



**UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE RUSSAS**

**CURSO: CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: LINGUAGENS FORMAIS E AUTÔMATOS
PROF. DR. CENEZ ARAÚJO DE REZENDE**

DESENVOLVIMENTO E ADAPTAÇÃO DE AUTÔMATOS COM PILHA (PDA): IMPLEMENTAÇÕES E DESAFIOS EM JAVA E PYTHON

Gabriel Miranda Gomes Cerqueira - 511916
Marco Antonio Rodrigues Gomes de Souza - 511354
Natam Leão Ferreira - 516205

RELATÓRIO

1.1. Quais itens de atividades foram importantes em termos teóricos e práticos?

A implementação da linguagem “enquanto” e do NPDA (Autômato com Pilha Não Determinístico) revelou-se extremamente proveitosa em termos de aquisição de conhecimento. Esse processo não só destacou os principais gaps no conhecimento teórico, mas também ofereceu uma oportunidade valiosa para aprofundar a experiência prática. Através dessas atividades, foi possível integrar melhor a teoria com a prática e identificar áreas que exigem maior aprofundamento, resultando em um crescimento significativo tanto no entendimento conceitual quanto na aplicação prática.

1.2. Das atividades que conseguiu concluir, classificaria como fácil, mediano ou difícil?

A implementação da linguagem 'enquanto' apresentou desafios, pois inicialmente foi realizada com apenas três estados, o que parecia não fazer muito sentido. Com a ajuda de exemplos e vídeos do professor, foi possível identificar e corrigir esses erros. Em geral, a implementação foi considerada mediana.

Por outro lado, a implementação do NPDA foi bastante complicada e não foi possível concluí-la sozinha. Foi necessária a ajuda de ferramentas de IA para resolver alguns problemas que surgiram ao longo do caminho, e também foi feito um estudo do código-fonte da biblioteca utilizada no Java, que foi considerado difícil.

Inicialmente, procurou-se implementar o NPDA com o auxílio de IA, mas surgiram dificuldades no tratamento da pilha. O método interativo apresentou problemas ao não conseguir verificar dois caminhos simultaneamente e ao não encontrar transições entre o topo da pilha e o primeiro caractere da palavra, pois o topo da pilha sempre era o caractere “#”.

Então, optou-se pela abordagem recursiva, que mostrou correção apenas para a linguagem que definia os binários múltiplos de três, mas não para outros testes. Após abandonar essa abordagem, foi realizada engenharia reversa no NPDA em Java já existente. Com essa abordagem, o algoritmo final se mostrou correto em todos os testes.

1.3. Das que não conseguiu concluir, quais foram as dificuldades encontradas?

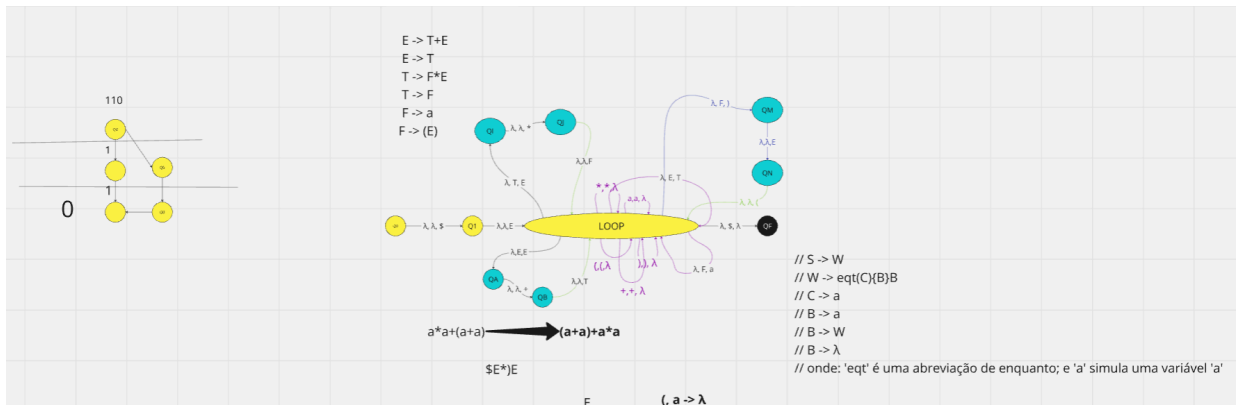
Foram completadas todas as atividades propostas, mas encontraram-se dificuldades na implementação do 'enquanto', principalmente por haver dificuldades em entender o funcionamento do loop. Então, antes de começar a escrever o código, os slides foram estudados para se ter uma melhor compreensão, fez-se engenharia reversa do código disponibilizado, utilizaram-se os logs gerados durante a execução dos testes e criou-se um quadro interativo, onde foi desenhada e desenvolvida a lógica antes de aplicá-la. Cada um desses passos foi importante para o resultado.

Também houve dificuldade na implementação do NPDA. Inicialmente, procurou-se implementá-lo com o auxílio de IA, mas surgiram problemas significativos no tratamento da pilha. O método interativo apresentou dificuldades ao não conseguir verificar dois caminhos simultaneamente e ao não encontrar transições entre o topo da pilha e o primeiro caractere da palavra, já que o topo da pilha sempre era o caractere “#”. A abordagem recursiva também enfrentou problemas, mostrando correção apenas para a linguagem que definia os binários múltiplos de três, mas não para outros testes. Finalmente, após essas dificuldades, foi realizada engenharia reversa no NPDA em Java já existente, que se revelou correto em todos os testes.

1.4. Utilizou IA para gerar alguma codificação? Caso, sim, necessitou de ajustes no código gerado, como foi a experiência?

Utilizou-se IA para auxiliar na criação do código. Inicialmente, gerou-se um código para resolver a proposta geral do problema apresentado e iniciou-se um processo de testes e alterações, já que a IA não conseguiu prever alguns dos bugs que surgiram. Por exemplo, durante a implementação do 'enquanto', o código gerado pela IA não retornava certos estados para o loop. Foi somente mediante um processo de refinamento do código que a equipe conseguiu superar o desafio.

ANEXOS



$q0[0]: [\#] [q1]\text{eqt}(a)\{\text{eqt}(a)\}$ $q1[0]: [\#\$] [q2]\text{eqt}(a)\{\text{eqt}(a)\}$ $q2[1]: [\#\$] e[q3]qt(a)\{\text{eqt}(a)\}$ $q3[2]: [\#\$] eq[q4]t(a)\{\text{eqt}(a)\}$ $q4[3]: [\#\$] \text{eqt}[q5](a)\{\text{eqt}(a)\}$ $q5[4]: [\#\$] \text{eqt}([q5]a)\{\text{eqt}(a)\}$ $q5[5]: [\#\$] \text{eqt}(a[q6])\{\text{eqt}(a)\}$ $q6[6]: [\#\$] \text{eqt}(a)[q6]\{\text{eqt}(a)\}$ $q6[7]: [\#\$] \text{eqt}(a)\{[q1]\text{eqt}(a)\}$ $q1[8]: [\#\$] \text{eqt}(a)\{e[]qt(a)\}$ $q1[8]: [\#\$] \text{eqt}(a)\{e[]qt(a)\}$ Não reconheceu: $\text{eqt}(a)\{\text{eqt}(a)\}$	$q0[0]: [\#] [q1]\text{eqt}(a)\{\text{eqt}(a)\}$ $q1[0]: [\#\$] [q2]\text{eqt}(a)\{\text{eqt}(a)\}$ $q2[1]: [\#\$] e[q3]qt(a)\{\text{eqt}(a)\}$ $q3[2]: [\#\$] eq[q4]t(a)\{\text{eqt}(a)\}$ $q4[3]: [\#\$] \text{eqt}[q5](a)\{\text{eqt}(a)\}$ $q5[4]: [\#\$] \text{eqt}([q5]a)\{\text{eqt}(a)\}$ $q5[5]: [\#\$] \text{eqt}(a[q6])\{\text{eqt}(a)\}$ $q6[6]: [\#\$] \text{eqt}(a)[q6]\{\text{eqt}(a)\}$ $q6[7]: [\#\$] \text{eqt}(a)\{[q2]\text{eqt}(a)\}$ $q2[8]: [\#\$] \text{eqt}(a)\{e[q3]qt(a)\}$ $q3[9]: [\#\$] \text{eqt}(a)\{eq[q4]t(a)\}$ $q4[10]: [\#\$] \text{eqt}(a)\{\text{eqt}[q5](a)\}$ $q5[11]: [\#\$] \text{eqt}(a)\{\text{eqt}([q5]a)\}$ $q5[12]: [\#\$] \text{eqt}(a)\{\text{eqt}(a[q6])\}$ $q6[13]: [\#\$] \text{eqt}(a)\{\text{eqt}(a)[q6]\}$ $q6[14]: [\#\$] \text{eqt}(a)\{\text{eqt}(a)\{[q7]\}$ $q7[15]: [\#\$] \text{eqt}(a)\{\text{eqt}(a)\}\}$ $q7[15]: [\#\$] \text{eqt}(a)\{\text{eqt}(a)\}\}$ Não reconheceu: $\text{eqt}(a)\{\text{eqt}(a)\}$	$q0[0]: [\#] [q1]\text{eqt}()\{\text{eqt}(a)\}$ $q1[0]: [\#\$] [q2]\text{eqt}()\{\text{eqt}(a)\}$ $q2[1]: [\#\$] e[q3]qt()\{\text{eqt}(a)\}$ $q3[2]: [\#\$] eq[q4]t()\{\text{eqt}(a)\}$ $q4[3]: [\#\$] \text{eqt}[q5]()\{\text{eqt}(a)\}$ $q5[4]: [\#\$] \text{eqt}([q6])\{\text{eqt}(a)\}$ $q0[0]: [\#] [q1]\text{eqt}()\{\text{eqt}(a)\}$ $q1[0]: [\#\$] [q2]\text{eqt}()\{\text{eqt}(a)\}$ $q2[1]: [\#\$] e[q3]qt()\{\text{eqt}(a)\}$ $q3[2]: [\#\$] eq[q4]t()\{\text{eqt}(a)\}$ $q4[3]: [\#\$] \text{eqt}[q5]()\{\text{eqt}(a)\}$ $q5[4]: [\#\$] \text{eqt}([q6])\{\text{eqt}(a)\}$ $q6[5]: [\#\$] \text{eqt}([q6])\{\text{eqt}(a)\}$ $q2[1]: [\#\$] e[q3]qt()\{\text{eqt}(a)\}$ $q3[2]: [\#\$] eq[q4]t()\{\text{eqt}(a)\}$ $q4[3]: [\#\$] \text{eqt}[q5]()\{\text{eqt}(a)\}$ $q5[4]: [\#\$] \text{eqt}([q6])\{\text{eqt}(a)\}$ $q6[5]: [\#\$] \text{eqt}([q6])\{\text{eqt}(a)\}$ $q6[6]: [\#\$] \text{eqt}([q2])\{\text{eqt}(a)\}$ $q6[5]: [\#\$] \text{eqt}([q6])\{\text{eqt}(a)\}$ $q6[6]: [\#\$] \text{eqt}([q2])\{\text{eqt}(a)\}$ $q2[7]: [\#\$] \text{eqt}([e[q3]qt(a)]\{\text{eqt}(a)\}$ $q2[7]: [\#\$] \text{eqt}([e[q3]qt(a)]\{\text{eqt}(a)\}$ $q3[8]: [\#\$] \text{eqt}([eq[q4]t(a)]\{\text{eqt}(a)\}$ $q4[9]: [\#\$] \text{eqt}([eqt[q5](a)]\{\text{eqt}(a)\}$ $q5[10]: [\#\$] \text{eqt}([eqt([q5]a)]\{\text{eqt}(a)\}$ $q5[11]: [\#\$] \text{eqt}([eqt(a[q6])]\{\text{eqt}(a)\}$ $q6[12]: [\#\$] \text{eqt}([eqt(a)[q6]]\{\text{eqt}(a)\}$ $q6[13]: [\#\$] \text{eqt}([eqt(a)\{[q7]\}\{\text{eqt}(a)\}$ $q7[14]: [\#\$] \text{eqt}([eqt(a)]\{[q7]\}\{\text{eqt}(a)\}$ $q7[15]: [\#\$] \text{eqt}([eqt(a)]\{[q7]\}\{\text{eqt}(a)\}$ $q7[15]: [\#\$] \text{eqt}([eqt(a)]\{[q7]\}\{\text{eqt}(a)\}$
--	---	---